Tech Science Press

ARTICLE

# AutoResearch-Ledger: Transactional Revision Integrity for Executable Research Agents

**Rajesh Kumar[1], Waqar Ali[2,*], Nikesh Lagun[3,4] and Shaoning Zeng[5]**

[1]International Research Center for Complexity Sciences, Hangzhou International Innovation Institute, Beihang University, Hangzhou, 311115, China
[2]College of Science, Mathematics and Technology, Wenzhou-Kean University, Wenzhou, China
[3]Hangzhou International Innovation Institute, Beihang University, Hangzhou, Zhejiang 311115, China
[4]Lagun Lab, Kathmandu, Nepal
[5]Yangtze Delta Region Institute (Huzhou), University of Electronic Science and Technology of China, Huzhou 313001, China
[*]Corresponding Author: Waqar Ali. Email: waqar.uestc@yahoo.com

Version October 27, 2026 submitted to Comput Mater Contin.

**ABSTRACT:** Persistent research agents can retain claims, decisions and manuscript text after the code, data or evidence on which they depend has changed. We present AutoResearch-Ledger (TRL), a transactional state-control layer for *declared* research dependencies. TRL represents conjunctive and alternative support with support sets, propagates lineage-aware invalidation through typed research objects, and applies a revision on a private copy before invariant checking and atomic commit or rollback. Its guarantees are explicitly conditional on the registered dependency graph: TRL checks consistency of recorded commitments and provenance, not scientific truth or evidentiary sufficiency. We evaluate three separable questions. First, on synthetic mechanism fixtures, full TRL matches an independently written oracle in 815/815 non-empty revision events; registration-corruption experiments show that missing, flattened and spurious relations can each defeat protection. Second, 13 genuine historical scientific-software revisions across nine repositories reproduced their documented semantic change and were carried through immutable old/new ResearchLedger runs, evidence supersession, manuscript provenance audit and recovery using developer-authored bindings. Third, local transaction fault injection passed 6/6 checks, including exact restoration at all 11 tested entity-write failure positions. In a separate authored 10-event structural pilot, support-set provenance and full TRL both achieved 10/10 exact impact identification, showing that selective impact identification comes from AND/OR support semantics, whereas the transaction layer contributes atomicity, rollback, freshness, recovery and auditable history. The integrated validation build passed 206/206 tests. We do not evaluate code generation (e.g., HumanEval, MBPP or SciCode), citation validity or end-to-end workflow completion, and we make no comparative claims about them. We also do not report independent human dependency annotation or native old/new executable-paper-agent replay; the present evidence establishes implementation and integration behavior under declared bindings, not end-to-end scientific validity of real research agents.

**KEYWORDS:** Research agents, provenance, transactional state, dependency propagation, truth maintenance, machine-checkable traceability.

© 2026 by the Authors. Submitted to Comput Mater Contin. for under a Creative Commons Attribution 4.0 International License, which permitted unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

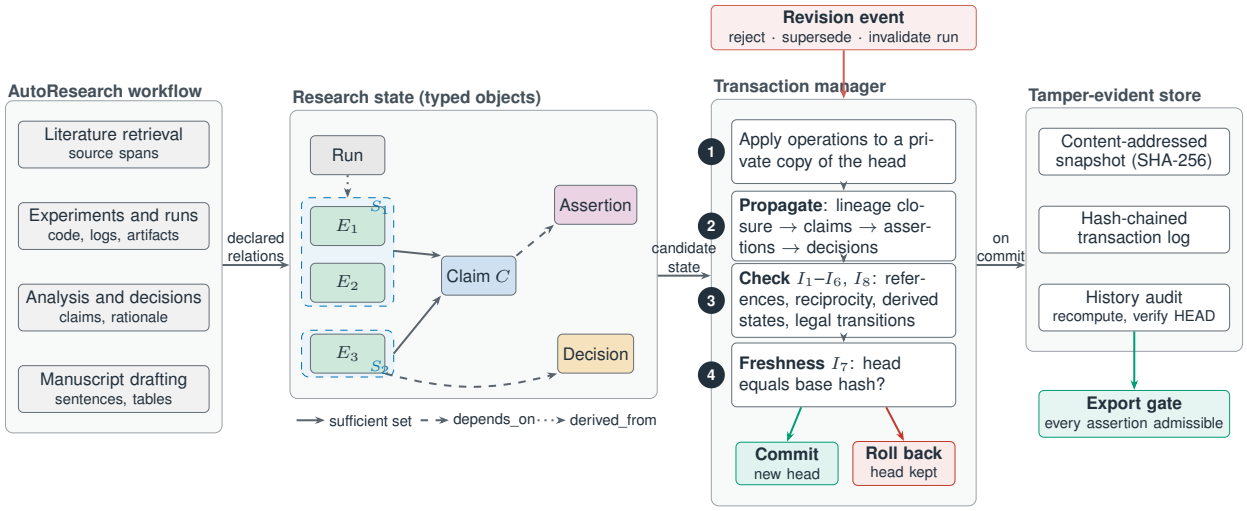



**Figure 1:** TRL architecture. Pipeline stages register typed relations among runs, evidence, claims, decisions and manuscript assertions. A revision event enters the transaction manager, which applies it to a private copy, propagates it through the lineage closure, checks invariants $I_1$–$I_8$, and either commits a content-addressed, hash-chained snapshot or rolls back. The export gate admits a manuscript only if every assertion is ledger-admissible under the registered graph.

## 1 Introduction

Research agents now search literature, formulate hypotheses, run code and draft manuscripts [1–4]. Paper2Agent extends this direction by converting papers, code, data and workflows into validated MCP-based executable agents that can reproduce analyses and answer new scientific queries [5]. Recent work has also shifted attention from generation to *verifiability*: ScientistOne maintains chains of evidence, EviGraph makes a typed evidence graph the agent's operational state, ARIS audits manuscript statements against a claim ledger, and claim–evidence provenance interfaces have been evaluated with researchers in their own right [6–9]. Agentification makes a remaining systems question more acute: after an upstream code, data, result or evidence correction, which persisted outputs, claims, decisions and manuscript statements remain current *relative to their registered dependencies*? A corrected run can change one artifact while downstream state remains unchanged, even when both old and new software execute successfully. We call this a *stale registered dependency*; the term concerns recorded state, not an independent judgement that the scientific conclusion is false.

We study this as a problem of *revision semantics* over a typed, versioned *registered research state* (Fig. 1). The Transactional Research Ledger (TRL) applies each change to a private copy, propagates it through the registered dependency graph $G$, checks global invariants, and either commits a content-addressed snapshot or rolls back. Its contract is deliberately conditional: if a dependency is absent from or mistyped in $G$, the transaction layer cannot infer it. Two design decisions carry most of the weight. First, a claim's dependence on evidence is not generally a set of independent edges: a claim that needs an experiment *and* a robustness check survives the loss of one only if a different sufficient set remains. TRL therefore represents claims by *support sets*. Second, the label "verified" is separated into what deterministic code can check (artifact, execution, result) and what it cannot (scientific validity), so that TRL never certifies more than it can inspect.

We do not claim novelty for evidence graphs, provenance, truth maintenance, optimistic concurrency or transactional agent state, nor for support sets themselves, which correspond to the monotone fragment of ATMS labels and positive-Boolean provenance (Section 2). The contribution is a research-specific *composition and operational contract*: typed recorded commitments, AND/OR dependency semantics, deterministic propagation, transactional activation, manuscript gating, and explicit measurement of dependency-registration quality. TRL is therefore an execution engine for supplied dependency declarations, not a method for discovering scientifically correct dependencies. The contributions are:

- **Selective revision semantics**: support sets, required premises, non-epistemic dependencies and lineage represent conjunction, alternatives and transitive derivation beyond flat singleton support edges (Section 3.2).
- **Transactional integrity**: typed lifecycles and invariants $I_1$–$I_8$ provide atomic commit/rollback, predecessor freshness and tamper-evident local history, with guarantees explicitly scoped to the registered graph (Section 3.3).
- **Operational construct boundaries**: provenance verification is separated from scientific validity, and ledger-liveness/admissibility are defined as graph-relative workflow states rather than truth values (Section 3.1).
- **Registration diagnostics**: typed precision, recall and support-set coverage distinguish missing, flattened and spurious relations, making the principal unprotected failure mode measurable (Sections 3.4, 5.3).
- **Layered validation**: synthetic mechanism/conformance experiments, 13 genuine historical revisions passed through a developer-authored tracked proxy, and separate transaction fault injection test different parts of the system without treating any of them as independent end-to-end scientific validation (Sections 4–5.7).
- **A preregistered external-validation instrument**: the remaining confirmatory study uses native old/new executable-paper-agent trajectories, blind annotation, matched logical baselines and repository-clustered uncertainty (Section 4).

Our questions are: **RQ1** does TRL preserve invariants and prevent stale claims *given a registered graph*; **RQ2** which mechanisms are necessary, including support sets and lineage; **RQ3** is its state machine-checkable (deterministic validation, rollback, tamper-evident history) and practical up to $10^5$ triples; **RQ4** how sensitive is protection to registration quality; **RQ5** can genuine historical semantic revisions be carried through immutable runs, evidence supersession, claim invalidation, manuscript audit and recovery in the implemented ledger; and **RQ6** how well do independently generated real paper agents register dependencies and preserve scientific state under genuine revisions. RQ1–RQ4 are answered with mechanism experiments, RQ5 with a developer-run tracked-proxy bridge study, and RQ6 and all human-audit claims remain open. Extended tables, protocols and proofs are in the Supplement (Supplement S1–S10).

## 2 Related Work

*Research agents and evidence assurance*

Coscientist, ChemCrow and the Virtual Lab couple LLM planning with tool use and experiment design or execution, and Agent Laboratory additionally connects experimentation to writing [1–4]. Paper2Agent converts a paper and its public computational assets into tested MCP tools and

reports robustness to several execution-level repository defects [5]. That addresses whether a paper agent can be made executable and kept available; our target is complementary: whether *persisted downstream research state* remains admissible after a scientifically consequential upstream revision, including a revision for which both versions still execute. ScientistOne requires generated claims to stay traceable to evidence across the lifecycle; EviGraph repairs downstream subgraphs when a weak dependency is found; ARIS audits manuscript statements against a claim ledger and raw evidence [6–8]. Claim–evidence provenance has also been studied as a user-facing auditing aid [9]; we do not evaluate it, as we ran no human study. None of these was run head-to-head with TRL; Supplement S8 tabulates their reported mechanisms. ScientistOne, EviGraph and ARIS (and the Continuity Kernel discussed below) are, at the time of writing, non-peer-reviewed preprints; we cite them only to position TRL against the closest contemporaneous systems and draw no evidential support from them.

*Truth maintenance, belief revision and argumentation*

Justification-based truth maintenance labels nodes IN or OUT from justifications and revises dependents when an assumption is withdrawn [10]; assumption-based maintenance instead labels each node with the minimal sets of assumptions (environments) under which it holds and answers queries in any context [11]. TRL's claim liveness is the monotone fragment of both: a sufficient set is an in-list, required premises and non-epistemic dependencies are in-list members common to every justification, lineage is the justification of derived evidence, and a claim's family of sufficient sets is its ATMS label. The same function is the positive-Boolean provenance polynomial of the claim over its evidence, and rejecting evidence is deletion propagation [12,13]. We test this correspondence (Section 5.5, Supplement S8). TRL lacks non-monotonic (OUT) justifications, nogood-directed backtracking and an ATMS's lattice of contexts; contradicting evidence only makes a claim `contested`. What it adds is orthogonal: atomic, predecessor-anchored commit and rollback, hash-chained history, typed lifecycles and a fail-closed export gate. AGM theory characterizes rational revision [14]; structured argumentation represents claims as conclusions of premise sets under attack relations [15–17]. Our support sets are a restricted, *declared* fragment: sufficient sets are premise sets, required premises are common to all of them, contradicting evidence is a rebutter. TRL neither weighs argument strength nor revises beliefs; it revises recorded commitments.

*Provenance, workflows and research objects*

Database provenance and W3C PROV model derivation [12,13,18]; scientific-workflow systems distinguish prospective from retrospective provenance [19,20]; Research Objects, RO-Crate, ReproZip and FAIR package artifacts for reuse [21–24]. They describe a package; TRL maintains its *state* under revision. A derivation record cannot say whether an input was necessary, sufficient or incidental to a claim; support sets can, but are declared, not captured.

*Transactions and propagation*

TRL's propagation is a restricted instance of incremental view maintenance [25], its freshness invariant an application of optimistic concurrency control [26], and predecessor-anchored activation is shared with the Continuity Kernel for generic agent state [27]. Build systems and self-adjusting computation likewise invalidate and recompute dependents of a changed input [28–30]. The

**Table 1:** Per-object state machines (full relations in Supplement S1). Claim and assertion states are derived from the support structure; the rest change by explicit operation.

| Object | States | Notes |
|---|---|---|
| Evidence | proposed, accepted, provenance_verified, rejected, superseded | `rejected` and `superseded` terminal: reviving a result needs a new record |
| Claim | unsupported, supported, verified, contested, stale | derived; stored state must equal the derivation ($I_2$) |
| Decision | proposed, decided, revisiting, revised, abandoned | decided/revised may not cite non-live evidence ($I_4$) |
| Assertion | draft, admissible, blocked, outdated | admissible/blocked derived from bound claims |
| Run | created, successful, failed, invalidated | failed, invalidated terminal |

difference is the object being maintained: TRL revises recorded commitments (claims, decisions and manuscript assertions) whose dependency semantics are declared and may include alternative sufficient paths, rather than merely recomputing a derived value. This is an architectural distinction, not a claim that invalidation, transactions or versioned state are new.

*Novelty boundary*

The closest formal account of TRL's support logic is ATMS/positive-Boolean provenance; the closest systems ingredients are provenance/workflow lineage, dependency invalidation and predecessor-anchored transactional activation. Accordingly, our novelty claim is limited to the *combination* of these mechanisms for typed research commitments and to its registration-aware evaluation. Table S11 compares the reported mechanisms of the closest research-agent systems, and Table S12 gives the exact correspondence to truth-maintenance/provenance models. "Not reported" in that comparison is descriptive, not evidence that another implementation could not support the feature. The survey is not exhaustive of all event-sourcing, data-versioning or ML-pipeline-lineage systems, so we make no universal absence claim outside the systems and formalisms analyzed here.

## 3 Transactional Research State

### *3.1 Research state and typed objects*

*Registered research state* is the set of recorded commitments—runs $U$, evidence records $E$, claims $C$, decisions $D$, manuscript assertions $A$—and the typed relations among them. It is not truth, belief probability or semantic warrant: TRL maintains the consistency of a *declared* structure under revision and does not judge whether the declaration is complete or scientifically justified. Throughout the paper, *live* and *admissible* mean *ledger-live* and *ledger-admissible with respect to the registered graph* $G$. Each object kind has a closed state set and legal transitions (Table 1); a committed transition outside the relation is rejected (invariant $I_8$). Labels are workflow states, not truth values, and `rejected` (evidence) and `stale` (claim) are deliberately not the same thing.

*Provenance-verified evidence*

We separate four things: *artifact verified* (declared artifacts exist and match their SHA-256), *execution verified* (the bound run succeeded and recorded an environment digest), *result verified* (a

**Table 2:** Operational scope of the central constructs. The middle column is machine-checkable once records and relations are supplied; the right column is an external scientific judgement that TRL does not infer.

| Construct | Operational meaning inside TRL | Outside TRL / requires external judgement |
|---|---|---|
| Evidence record | Versioned object with source/run/artifact identity and a workflow state | Whether the object is scientifically relevant or valid evidence for a proposition |
| Sufficient set $S \in \mathcal{S}(c)$ | All members must be ledger-live; one complete set is enough to keep $c$ live | Whether those records are in fact jointly sufficient for the scientific claim |
| Required premise $R(c)$ | Every member must remain ledger-live in addition to any sufficient set | Whether the item is genuinely necessary across all evidentiary routes |
| Non-epistemic dependency $D(c)$ | Loss forces re-examination/staleness but does not raise the provenance support level $\lambda$ | Whether a code, preprocessing, model or assumption change should invalidate the claim |
| Lineage | Declared `derived_from` relation whose liveness propagates transitively | Whether unrecorded or latent dependence exists |
| Assertion binding | Explicit span-to-claim relation used by the export gate | Text segmentation, binding granularity and whether the assertion is scientifically warranted |
| Ledger liveness and admissibility | Deterministic status derived from $G$, object states and invariants | Truth, credibility, effect size, evidentiary strength or reviewer acceptance |

declared metric is reproduced from the recorded artifact by a supplied checker), and *scientifically validated* (the experiment is valid evidence for the proposition). TRL evaluates the first three deterministically; a record is `provenance_verified` only if no tier failed and at least one passed, and a failed re-check demotes it. The fourth is deliberately outside the data model: a matching hash and a clean exit code do not make an experiment a valid test of a hypothesis. Accordingly a `verified` claim means only that every member of its best sufficient set is provenance-verified; it should be read as *provenance-complete*, not as true or well tested. Table 2 makes the same boundary explicit for the remaining constructs. The implementation retains the concise labels `live` and `admissible`; in interpretation they are always graph-relative workflow labels, never judgements of credibility or scientific warrant.

### *3.2 Support-set semantics*

Registering $E_1 \to C$ and $E_2 \to C$ as independent edges lets $C$ survive the loss of $E_1$ on the strength of $E_2$, even if $C$ needs both. We therefore derive for each claim $c$ a family $\mathcal{S}(c)$ of *jointly sufficient* evidence sets, necessary premises $R(c)$, and non-epistemic dependencies $D(c)$ (evidence the claim depends on without it counting as support). Conjunction is a set with several members;

**Table 3:** Relation vocabulary and its role in Eqs. (1)–(2).

| Relation | Semantics |
|---|---|
| `supports` | singleton sufficient set $\{e\} \in \mathcal{S}(c)$: positive, not individually necessary |
| `support_group` | one conjunctive set $S \in \mathcal{S}(c)$ |
| `alternative_support_group` | a further set in $\mathcal{S}(c)$ (sets in $\mathcal{S}$ are alternatives) |
| `required_for` | $e \in R(c)$: necessary in addition to whichever set is used |
| `contradicts` | negative evidence; makes a live claim `contested` |
| `depends_on` | $e \in D(c)$, or a decision/assertion dependency: invalidates when lost, never counts toward $\lambda$ |
| `derived_from` | lineage from evidence or a run to evidence; loss propagates transitively |

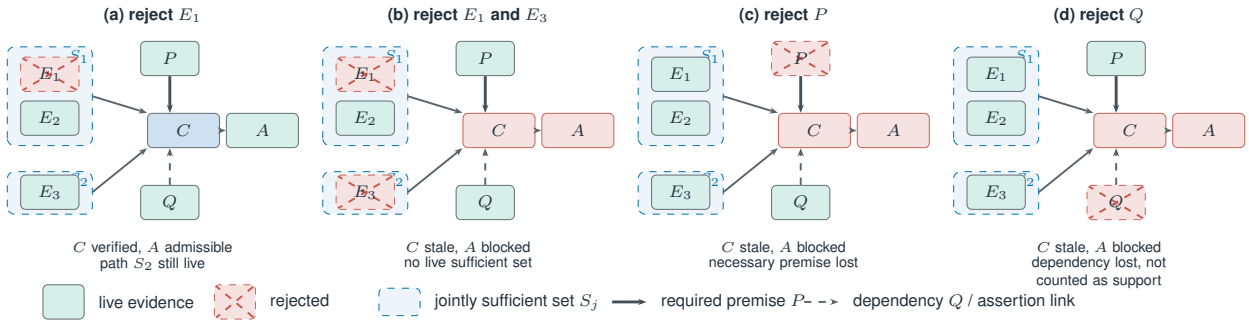



**Figure 2:** Support-set semantics: one claim $C$ with sufficient sets $S_1 = \{E_1, E_2\}$ and $S_2 = \{E_3\}$, a necessary premise $P$, a non-epistemic dependency $Q$ and a manuscript assertion $A$, under four revisions. (a) Losing $E_1$ breaks $S_1$ but $S_2$ survives. (b) Losing $E_1$ and $E_3$ leaves no sufficient set. (c) Losing $P$ stales $C$ although both sets are intact. (d) Losing $Q$ stales $C$ without $Q$ ever counting as support. Under binary edges only outcome (a) is reproduced; in (b)–(d) $C$ and $A$ would be left unchanged.

disjunction is several sets. Let $\mathrm{live}(e)$ hold iff $e$ is neither rejected nor superseded, was not produced by a failed or invalidated run, and every record in its `derived_from` lineage is live. Then

$$\mathrm{Live}(c) \iff \big(\exists S \in \mathcal{S}(c)\ \forall e \in S:\ \mathrm{live}(e)\big) \wedge \big(\forall e \in R(c) \cup D(c):\ \mathrm{live}(e)\big), \tag{1}$$

$$\lambda(c) = \max_{S \text{ live}} \min_{e \in S \cup R(c)} \ell(e), \tag{2}$$

where $\ell$ maps proposed, accepted and provenance_verified to $0, 1, 2$. The claim state is a deterministic function: `contested` if $\mathrm{Live}(c)$ and contradicting evidence is live; else `verified` ($\lambda = 2$), `supported` ($\lambda = 1$) or `unsupported` ($\lambda = 0$); if $\neg\mathrm{Live}(c)$, `stale` whenever a registered sufficient set, premise or dependency exists, so that something once registered has been lost. A claim's level is that of the weakest member of its best live set; $\lambda$ propagates provenance tiers and has no further epistemic interpretation. A claim citing only proposed evidence is `unsupported` (it never had a live sufficient path at level $\geq 1$), whereas one whose accepted evidence is later rejected is `stale`. Fig. 2 shows one claim under four revisions, and Table 3 maps the relation vocabulary onto this structure. The earlier binary-edge semantics is the special case with only singleton sets, $R = D = \emptyset$ and no lineage.

**Proposition 1:** *Under singleton-edge semantics* $\mathrm{Live}(c) = \bigvee_e \mathrm{live}(e)$*; for no edge set is this equal to* $\mathrm{live}(E_1) \wedge \mathrm{live}(E_2)$ *on all assignments.*

**Proof:** If $E_1$ is an edge, the disjunction holds when only $E_1$ is live while the conjunction fails; symmetrically for $E_2$; if neither is an edge the disjunction ignores one of them. □

The result is elementary; its point is that support sets are *strictly* more expressive, so losing a conjunction at registration time is a failure distinct from losing an edge.

*What the components mean, and what they do not*

By Eqs. (1)–(2), adding the members of $R(c)$ to every set in $\mathcal{S}(c)$ leaves Live and $\lambda$ unchanged (only the `stale/unsupported` label differs for a claim with premises but no sets), so $R(c)$ adds no expressive power. It is a normalization device whose value is registration robustness: "common to every alternative" is declared once and its omission/flattening becomes separately measurable. $D(c)$ differs from $R(c)$ only in not entering $\lambda$. The difficult step is therefore not evaluation of Eqs. (1)–(2) but *construct assignment*: deciding whether a replication is required or reinforcing, whether a statistical specification is a premise, a non-epistemic dependency or part of the claim, and whether apparently independent alternatives share latent dependence. TRL takes those judgements as input. Evidential support is binary: there is no partial support, evidence-strength weighting or statistical dependence between nominally alternative sets. A manuscript assertion is a span explicitly bound to one or more claims; TRL does not segment text or decide binding granularity, so manuscript protection is conditional on binding quality. Supplement S7 provides a counterfactual annotation codebook and preregistered agreement test; because no independent annotation round has been completed, $R(c)$, $D(c)$ and support-set membership should be read as operational categories whose inter-rater construct validity remains open.

### *3.3 Invariants and transactions*

A candidate state is *ledger-admissible* only if it satisfies the following invariants.

- $I_1$ **Referential integrity:** every referenced run, evidence, claim, decision or assertion exists.
- $I_2$ **Reciprocity and derivation:** every relation is recorded on both endpoints, every support-set member is indexed on its claim, and each claim and assertion state equals its derivation.
- $I_3$ **Supersession and lineage closure:** superseded evidence names an existing replacement; supersession and lineage chains are acyclic.
- $I_4$ **Decision provenance:** a decision has evidence or a rationale, and a decided or revised decision cites no non-live evidence (including through lineage or run invalidation).
- $I_5$ **Artifact and verification integrity:** artifact digests match; evidence is not provenance-verified while a recorded tier failed or its run is unfinished.
- $I_6$ **Manuscript admissibility:** manuscript-referenced claims are supported or verified; every non-outdated assertion is ledger-admissible under its registered bindings.
- $I_7$ **Predecessor freshness:** the ledger head still equals the predecessor hash the transaction was built from.
- $I_8$ **Transition legality:** every object moves only along its state machine (Table 1).

$I_1$–$I_6$ and $I_8$ are deterministic checks by ordinary code; $I_7$ is optimistic concurrency control under a local commit lock. An LLM may propose a mutation but cannot override a failed check.

A transaction $\tau = (b, \Delta, a, r)$ carries the exact predecessor hash $b = h(S_t)$, operations $\Delta$, actor $a$ and rationale $r$. It is applied to a private copy, $\widehat{S} = \mathrm{Apply}(S_t, \Delta)$, and with $H_t$ the head at activation,

$$S_{t+1} = \begin{cases} \widehat{S}, & H_t = b \ \wedge \ \bigwedge_{j \in \{1,\ldots,6,8\}} I_j(S_t, \widehat{S}) = 1, \\ S_t, & \text{otherwise.} \end{cases} \tag{3}$$

A commit writes a content-addressed snapshot $h_{t+1} = \mathrm{SHA256}(S_{t+1})$ and a hash-chained log record with actor, rationale, operations and before/after hashes. The chain is tamper-*evident* against partial local edits; without an external signature an adversary with full write access could recompute it. Commit-safety (every committed state satisfies the invariants; a stale candidate cannot overwrite a newer head) follows by induction from Eq. (3) and is stated for completeness, not as a deep result.

**Proposition 2:** *Let $G$ be the registered graph and $E_0$ the evidence rejected or superseded in one transaction. After commit every claim, assertion and decision has the state that Eqs.* (1)–(2) *and the state machines assign on $G$ with liveness recomputed on the lineage closure of $E_0$, and nothing outside that closure's dependents changes.*

*Sketch.* Propagation reconciles exactly the claims linked to a closure member, the assertions bound to them and the decisions citing a member; nothing else is related to a record whose liveness changed. $I_2$ then rejects the commit unless every stored state equals its derivation.

The result is sound and complete *with respect to* $G$ and silent about the true dependency structure; its content is largely definitional. What is not definitional is the choice of semantics (Proposition 1) and how well $G$ is registered, both tested below. A replacement for superseded evidence does not inherit its support relations; they must be re-established explicitly.

### *3.4 Dependency registration*

Everything above operates on the registered graph, so registration is the principal source of unprotected failure. Writing $G = \mathcal{A}(T)$ for the graph an acquisition process $\mathcal{A}$ (a stage hook, a rule, a person or an LLM) derives from a trajectory $T$, TRL specifies only the revision step applied to $G$; its guarantees (Proposition 2) are conditional on $G$, and end-to-end protection cannot exceed the fidelity of $\mathcal{A}$. We do not implement automatic relation discovery or evaluate $\mathcal{A}$ on independently annotated real trajectories. TRL instead provides explicit registration channels—pipeline hooks called by the stage that knows the semantics, an adapter for ResearchLedger workspaces, and run capture that binds evidence to immutable runs and artifacts—and preserves registration provenance. For the confirmatory study, $G^\star$ is an *adjudicated reference graph*, not assumed ground truth by fiat: blind annotator agreement is measured first, and low agreement is itself evidence that the vocabulary is not sufficiently well defined. Conditional on an adjudicated $G^\star$, we define typed precision and recall $\mathrm{P}_{\mathrm{dep}} = |G \cap G^\star|/|G|$ and $\mathrm{R}_{\mathrm{dep}} = |G \cap G^\star|/|G^\star|$ (group and alternative-group edges interchangeable), *object coverage* (reference dependency-bearing objects with every incoming reference edge registered) and *support-set coverage* (reference claims whose full family of sufficient sets and premises is reproduced). Two failures that $\mathrm{R}_{\mathrm{dep}}$ hides are counted separately: a *flattened*

*conjunction* (a group or premise registered as independent `supports`: no missing edge, wrong semantics) and a *spurious edge* (which can disable protection by creating a fake alternative path).

## 4 Implementation and Evaluation Design

### *4.1 Implementation*

TRL is a model-agnostic Python layer (typed dataclasses, JSON, SHA-256, a JSONL log) around AutoResearch's 23-stage workflow; pipeline hooks record evidence, claims, decisions and manuscript bindings, and a fail-closed bridge to ResearchLedger supplies immutable runs, artifact hashing, tracing and manuscript audit (Supplement S2). The core specification in Table 1 and the ResearchLedger v2.2 bridge use different native status vocabularies. In particular, the tracked-proxy bridge reports ResearchLedger's `supported`→`hypothesis`→`supported` sequence; `hypothesis` there means that no currently sufficient live support path remains and is not a Table 1 TRL claim label. We retain native bridge labels rather than silently remapping them; Supplement S2 documents the semantic correspondence. The integrated v2.2 validation build—including support-set revision, transaction recovery, tracked trajectories, paper audit and study tooling—passed 206/206 automated tests across 22 modules in non-overlapping batches; no lint or static-type result is claimed for that run because those executables were unavailable.

### *4.2 Evidence hierarchy*

The evaluation separates four evidentiary questions that are often conflated: (i) whether the formal/state semantics are internally coherent, (ii) whether the implementation realizes those semantics, (iii) whether independent experts can map real research trajectories into the proposed dependency vocabulary, and (iv) whether using the system improves end-to-end revision behavior for real paper agents or human auditors. The current paper provides substantial evidence for (i)–(ii), bridge evidence toward (iii), and protocols rather than results for the decisive independent tests in (iii)–(iv) (Table 4).

### *4.3 Confirmatory old/new paper-agent benchmark (specified, not executed)*

The scientific validation this work needs has reference judgements outside TRL. The preregistered confirmatory study targets at least 60 genuine semantic revision events from at least 15 repositories (preferred 100–150 events from 25+ repositories). For each event, the old commit is frozen, an executable paper agent is generated where feasible (Paper2Agent provides one concrete standardized substrate [5]), a prespecified task is executed, and its outputs and downstream claims are persisted before the historical correction is replayed. Two annotators, blind to registrations and system predictions, independently label relations and affected claims, decisions and assertions; a third adjudicates. *Construct validity is a gate, not an assumption*: agreement on edge presence, relation type and support structure is reported before system accuracy, and the preregistration treats low agreement as evidence that the relation vocabulary itself requires refinement. Primary system outcomes are stale-assertion escape and false invalidation; secondary outcomes include claim precision/recall, selective recomputation, executions avoided relative to full rebuild, recovery correctness, time-to-safe-state, dependency precision/recall and support-set coverage. The preregistered matched-registration systems are: no revision-state management; invalidate-all/full rebuild; downstream reachability; flat binary

**Table 4:** Evidence hierarchy and current execution status. "Construct validity" asks whether independent annotators can reproducibly identify the relations that TRL assumes as input; "end-to-end utility" asks whether real agents benefit under genuine revision.

| Question | Current evidence | Status |
|---|---|---|
| Formal and implementation conformance | mechanism diagnostics, TMS cross-check, validator tests, scaling | executed |
| Transactional integrity | rollback, interrupted recovery, stale-writer protection, local history checks | 6/6 executed |
| Bridge/ecological plausibility | 13 genuine revision identities and old/new semantics carried through developer-authored proxy trajectories | 13/13 executed |
| Construct validity | blind relation/impact annotation and adjudication on real trajectories | not executed |
| End-to-end agent validity | native old/new executable-paper-agent replay with matched/native registration | not executed |
| Human auditability | time-to-locate and stale statements missed | protocol only |
| Code-generation, citation-validity and end-to-end workflow benchmarks | none; HumanEval, MBPP, SciCode, citation-validation and workflow-completion outcomes are outside the scope of this study | not evaluated |

dependency propagation; AND/OR support-set provenance without transactions; and full TRL. Where feasible, an ATMS or positive-Boolean-provenance implementation is added as a strong formal comparator. Repository, not event, is the resampling unit for 95% bootstrap intervals. The original EviGraph reconstruction remains a synthetic mechanism comparator only and is not treated as evidence about the native system. Protocols and the human-audit design are in Supplement S7; the executed 13-case tracked-proxy study below exercises the downstream ledger lifecycle but not real paper-agent generation or independent annotation.

### *4.4 Mechanism experiments*

**Conformance.** RSM-1000 (single mutations of a 20-claim state), RSM-SEQ-2500 (25 cumulative mutations $\times$ 100 episodes), RSR-500 (stale-writer races), HT-250 (five classes of local-history corruption), RLV2-400 (seven documented ResearchLedger rule classes) and DRS-1400 (unregistered dependencies) are rerun on the revised core with their original seeds (Supplement S3). **SUP-300 and REG.** A generator builds trajectories with five claim templates (singleton 30%, conjunctive 25%, alternative groups 20%, required premise 15%, dependency 10%), lineage, decisions and assertions; three evidence rejections per trajectory yield 815 events with non-empty impact (seed 61). REG corrupts the registered graph by dropped edges, flattened conjunctions or spurious edges (100 trajectories per point, seed 62). The generator authors the gold structure, so these are mechanism diagnostics whose template mix is arbitrary; only direction and mechanism are informative. **SM-CONF.** An exhaustive check of all 107 ordered state pairs, fuzzing with

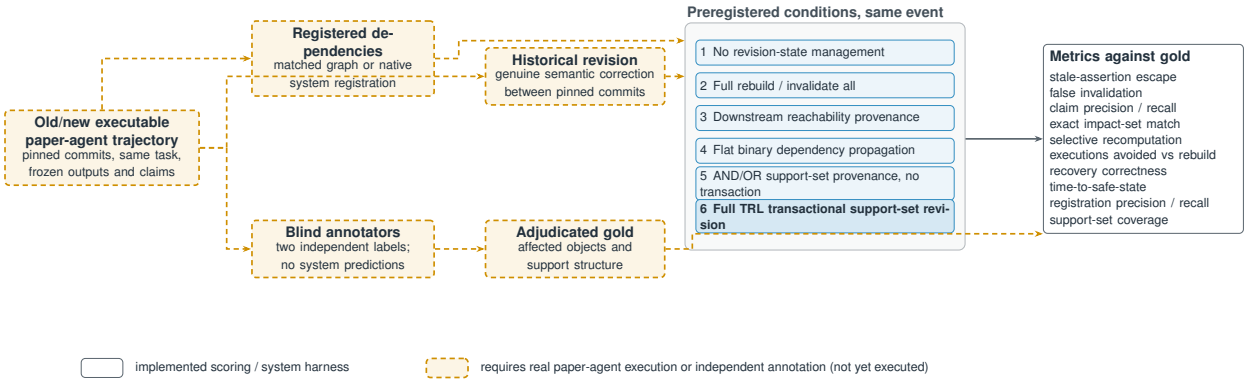



**Figure 3:** Confirmatory evaluation design. Dashed boxes require real old/new executable-paper-agent trajectories and blind human annotation and have not been executed. The 13-case tracked-proxy bridge study exercises the downstream immutable-run, invalidation, manuscript-audit and recovery path, but does not replace these external acquisition and annotation stages.

**Table 5:** Conformance benchmarks on the revised core (full tables in Supplement S3). Residual violations and escape are for the state after all mutations.

| Benchmark | Full TRL | Strongest non-TRL condition |
|---|---|---|
| RSM-1000 | 0 violations, escape 0%, all legitimate updates committed, all corrupt ones rejected | post-hoc repair: 1.206 violations/trial; raw state: escape 79.9% |
| RSM-SEQ-2500 | 100/100 episodes consistent | post-hoc repair: 30.0 violations at episode end |
| RSR-500 | OCC + rebase: both updates kept in 500/500 | last-write-wins: lost update in 500/500 |
| HT-250 | 250/250 corruptions detected, 0/25 false positives | — |
| RLV2-400 | 350/350 corruptions rejected with target rule, 50/50 clean accepted | — |
| DRS-1400 | ledger escape 0% at every rate; ground-truth escape $= p_{\text{missing}}$ | — |

random legitimate operations, and fuzzing with forbidden direct writes. **Scaling.** $10^3$–$10^5$ triples at 1, 5 and 20 links per claim, with component timing.

## 5 Results

### *5.1 Conformance benchmarks reproduce*

Table 5 summarizes the earlier benchmarks. Their result files are identical to the previous prototype's apart from timing fields; claim states now use the typed vocabulary, which changes no count. These results establish that the implementation matches its specification, not that the specification captures real revision. DRS-1400 makes one boundary explicit: when a true dependency is never registered, TRL raises no violation and ledger-reported escape stays at 0%, while ground-truth escape equals the unregistration rate.

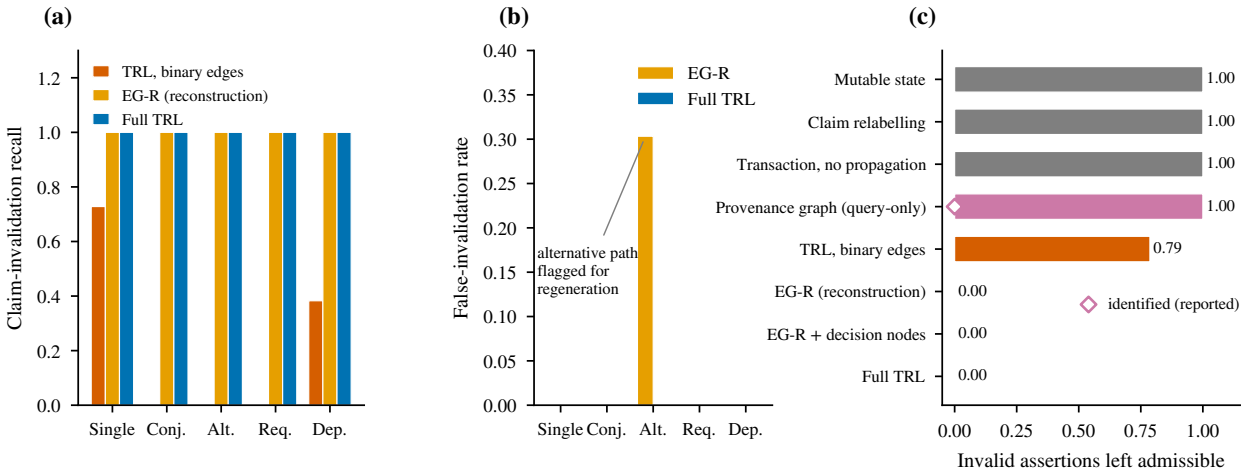



**Figure 4:** SUP-300 on synthetic trajectories with perfect registration (815 events). (a) Claim-invalidation recall by claim template: single, conjunctive, alternative-group, required-premise, non-epistemic dependency. (b) False invalidation of still-justified claims: only the reconstruction of EviGraph's mechanism over-invalidates, and only where an alternative sufficient path exists. (c) Invalid assertions left admissible, by condition; the diamond marks what the query-only provenance graph *reports* rather than updates.

### *5.2 Support sets and lineage (SUP-300)*

*Synthetic fixtures with generator-authored gold; magnitudes depend on the arbitrary template mix.* With perfect registration, full TRL's set of invalidated claims equalled the oracle's in all 815 events, and it recovered every affected decision and assertion (Fig. 4, Table 6). This is specification-level agreement on generator-authored graphs. It supports implementation correctness and the expressiveness claim for AND/OR dependencies, but it is not an estimate of real-world impact-identification accuracy.

These are representation sanity checks, not comparative evidence: the generator includes templates that singleton edges cannot express, so their failure is by construction. The other conditions fail in ways that map to semantic features. Claim relabelling and TRL without support sets recover 20.5% of invalidated claims: none of the conjunctive, alternative-group or required-premise claims, and 72.8% of singleton claims (the rest are lineage taint). A provenance store without transactions *identifies* every affected object but updates none, so all invalid assertions remain admissible (escape 1.00 by state, 0.00 by report). The EviGraph reconstruction recovers every affected claim and assertion but flags 30.3% of still-justified claims that have an alternative sufficient path, because flagging a whole subgraph cannot tell a lost path from a lost claim; its decision handling depends on whether decisions are graph nodes. Its over-invalidation costs needless regeneration rather than unsound state, and is favored by the given-invalid-node assumption. EG-R is a synthetic reachability baseline: the original system is reported to include further components (for example semantic consistency checks and graph checkpointing) that are not modelled, and no TRL-versus-EviGraph conclusion follows.

### *5.3 Registration sensitivity (REG)*

Fig. 5 shows escape of full TRL as the registered graph is corrupted; all observations are on synthetic fixtures. *Escape rises much faster than edge recall falls*: dropping 5% of true edges gives

**Table 6:** SUP-300 by condition. Recall: invalidated claims recovered; FIR: still-justified claims wrongly invalidated; Escape: invalid assertions left admissible. Values are over stored state unless marked "reported".

| Condition | Recall | FIR | Dec. recall | Escape |
|---|---|---|---|---|
| Mutable state | 0.000 | 0.000 | 0.000 | 1.000 |
| Claim relabelling | 0.205 | 0.000 | 0.000 | 1.000 |
| Provenance graph, updated | 0.000 | 0.000 | 0.000 | 1.000 |
| Provenance graph, reported | 1.000 | 0.070 | 1.000 | 0.000 |
| Transaction, no propagation | 0.000 | 0.000 | 0.000 | 1.000 |
| EG-R (reconstruction) | 1.000 | 0.070 | 0.000 | 0.000 |
| TRL, binary edges | 0.205 | 0.000 | 0.757 | 0.787 |
| **Full TRL** | **1.000** | **0.000** | **1.000** | **0.000** |

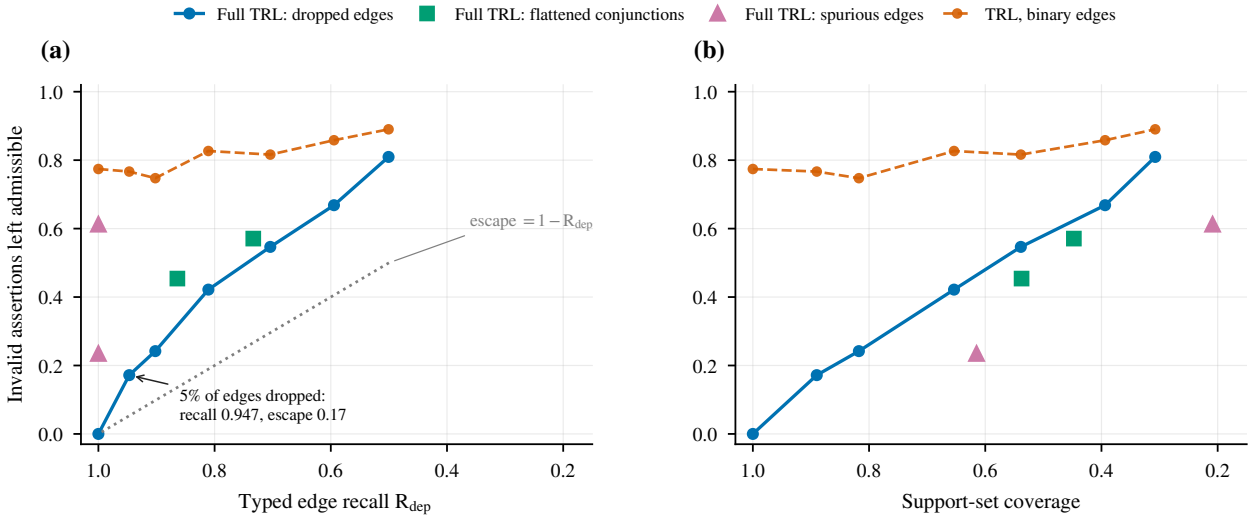



**Figure 5:** Protection under registration corruption (100 synthetic trajectories per point). Escape is the fraction of invalid manuscript assertions left admissible. (a) Against typed edge recall; the dotted line is escape $= 1 - \mathrm{R}_{\mathrm{dep}}$. (b) Against support-set coverage. Dropped edges, flattened conjunctions and spurious edges are separate corruption processes; the dashed line is TRL restricted to binary edges.

$\mathrm{R}_{\mathrm{dep}} = 0.947$ and support-set coverage 0.890, yet 17.2% of invalidated assertions escape; at 10% dropped, recall is 0.902 and escape 24.2%. Losing one member of a conjunction removes the whole claim from protection, and one lost lineage edge can do the same downstream. Over the seven drop rates escape $= 0.074 + 1.52\,(1 - \mathrm{R}_{\mathrm{dep}})$ ($R^2 = 0.977$, aggregate points, $n = 7$; a description of these fixtures, not a law), which is why edge recall alone is an insufficient registration metric. *Flattening* produces no missing edge yet yields 57.1% escape when every conjunction is flattened. *Spurious* edges with perfect recall still raise escape (23.4% at 10%, 61.2% at 30%): an extra `supports` edge creates a fake alternative path that keeps an invalid claim alive, so registration precision matters for protection, not only recall. TRL without support sets fails at every point.

### *5.4 State-machine conformance (SM-CONF)*

The exhaustive matrix agreed with the documented relations on all 107 ordered pairs. In 150 fuzzing episodes (seed 53), 4,582 of 4,611 operations committed with no invariant violation, *none* of the legitimate operations was rejected as an illegal transition, and the 29 rejections were all the intended `stale_decision`. The fuzzer exercised 44 of 48 table edges; all 500 forbidden direct

writes and 100 direct writes contradicting the derivation were rejected. The informative outcome is what the first pass did to the design: our initial claim machine declared `stale` sticky and forbade `supported`→`unsupported`, but re-supporting a stale claim with unaccepted evidence and declaring an unaccepted required premise reach exactly those transitions. We corrected the specification: claim states are derived and the operative constraint is agreement with the derivation. SM-CONF tests consistency of the implementation with its own tables, not that they are the right tables for any workflow.

### *5.5 Truth-maintenance cross-check (TMS-XC)*

On 300 synthetic trajectories (seed 61; 900 events per graph) we recomputed each revision's affected objects with the oracle, a Doyle-style JTMS, a de Kleer-style ATMS and full TRL, on the gold graph and on graphs with dropped, flattened or spurious edges. JTMS and ATMS matched the oracle on all 3,600 events; TRL matched on 3,551. Its 49 divergences all have mechanical causes (Supplement S8): 25 are assertions already blocked by an unsupported claim, which undergo no state transition, and 24 are mistyped duplicate edges on one (evidence, claim) pair, which collide in TRL's one-relation-per-pair model and make it over-invalidate. This is agreement among implementations of one semantics, not evidence about real research; a relation model with several relations per pair would remove the collision.

### *5.6 Scaling*

Fig. 6 shows one revision on a committed archive of $10^4$ to $10^5$ triples (three records each). Timings are from a noisy 24-thread Windows laptop (the earlier prototype's code is $\approx 3.5\times$ slower here than its published numbers, and medians at $10^3$ triples varied by up to $3\times$), so only scales $\geq 10^4$ are interpreted. Three findings are robust. *Propagation is not the bottleneck*: 5–30 ms through $5 \times 10^4$ triples and 0.2 s at $10^5$, under 1% of a revision. *Everything else is linear in archive size, and the full-snapshot design is the bottleneck*: a revision takes about 2 s at $10^4$ triples, 9–18 s at $5 \times 10^4$ and 65 s at $10^5$, with a 72 MB snapshot rewritten per commit. *Storage flushing is not the cost at scale*: with `fsync` disabled, commit time at $\geq 5 \times 10^4$ triples is unchanged. A single-pass store (one serialization, no deep copy) keeps hashes identical, roughly halves revision time and cuts snapshots by 37%, but changes constants, not asymptotics. Density matters less than record count (at $10^4$ triples, 1 to 20 links per claim raises a revision from 2.0 to 2.5 s). We did not run $5 \times 10^5$ triples; supporting that regime needs a persistent Merkle or delta structure with lazy loading and incremental validation, which we describe but do not implement. Full tables are in Supplement S6.

### *5.7 Historical revisions, tracked proxy trajectories and fault injection (development evidence)*

The source audit identified 13 genuine semantic revisions across nine scientific-software repositories (`POP-TOOLS`, `scanpy`, `squidpy`, `TabPFN`, `SAELens`, `Compass`, `scvi-tools`, `seurat`, `scikit-learn`). Replaying the smallest changed semantic unit reproduced the documented old/new behavioral, numerical, data or configuration difference in 13/13 cases (Supplement S9). The cases include sparse normalization, a statistical-variance correction, calibration under majority downsampling, feature subsampling, sequence-boundary handling, PCA setup, duplicated data removal and scientific defaults. These are narrow semantic replays rather than full historical repository builds.

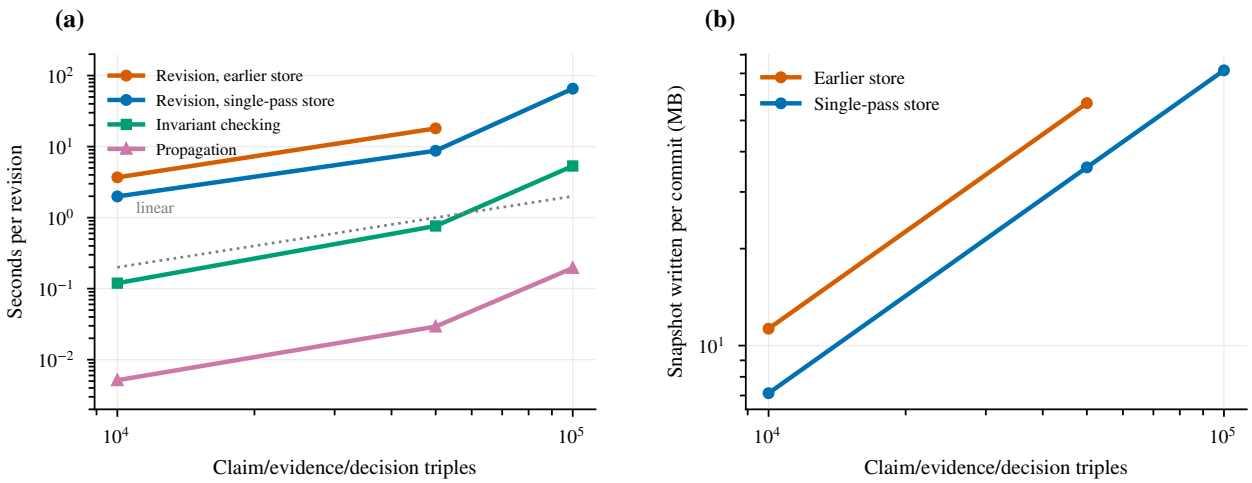



**Figure 6:** Scaling of a single evidence-rejection revision, $10^4$–$10^5$ triples (median of 3–5). (a) Time per revision, invariant checking and propagation under the earlier and the single-pass store; the dotted line has unit log-log slope. (b) Snapshot size, which is also the number of bytes written per commit.

We then passed every case through a deterministic paper-agent proxy as two separate immutable ResearchLedger runs. The old output was frozen as verified bridge evidence for a claim; the historical revision superseded that evidence transactionally; and the revised output was attached as replacement evidence and verified. All 13 trajectories produced different old/new artifact hashes and completed the same expected ResearchLedger-native lifecycle (Table 7): the claim was `supported` before revision, became `hypothesis` after the old evidence was superseded, and returned to `supported` after replacement verification. As noted in Section 4, these are bridge-layer labels: `hypothesis` denotes absence of a currently sufficient live support path and should not be confused with the core TRL labels in Table 1. An explicit manuscript provenance marker naming the superseded evidence was flagged stale in 13/13 cases and recovered in 13/13 after the marker was changed to the replacement evidence. Every final workspace had zero validation errors. Median local transaction-only time to the safe invalidated state was 9.6 ms and median recovery bookkeeping after the revised output already existed was 15.0 ms; old/new proxy tool runs were about 2.9–3.0 s and were dominated by local process/environment capture. These timings are implementation diagnostics, not Paper2Agent or production benchmarks.

We separately fault-injected the multi-file revision transaction under six deterministic checks. All passed, including forced write failure and simulated mid-transaction crash with exact pre-image restoration, 11/11 tested entity-write fault positions on an 11-file revision, preservation of a claim when one of two alternative sufficient support paths remained live, and two independent concurrent revision processes both reaching the expected final state. On a separate authored 10-event structural pilot, support-set provenance without transactions and full TRL both reached 10/10 exact impact identification; flat binary edges reached 6/10; unconditional downstream reachability and invalidate-all/full rebuild reached 4/10 exact matches because they over-invalidated claims retaining an alternative sufficient support path. A repository-clustered bootstrap over eight repository clusters gave a median TRL-minus-binary-edge claim-recall difference of 0.40 (95% percentile interval $[0.111, 0.700]$) and a median TRL-minus-support-set-provenance exact-match difference of 0.0 ($[0, 0]$). Because the impact graphs and gold sets were authored for

**Table 7:** Ledger-integrated tracked proxy trajectories over 13 genuine historical revisions. Bindings and proxy tools were developer-authored; this is bridge/conformance evidence, not independent external validation.

| Outcome | Result |
|---|---|
| Historical revisions / repositories | 13 / 9 |
| Documented old/new semantic change reproduced | 13/13 |
| Old/new artifact hash changed | 13/13 |
| Claim `supported`→`hypothesis` after supersession | 13/13 |
| Explicit superseded-evidence manuscript marker flagged stale | 13/13 |
| Replacement evidence restored `supported` claim | 13/13 |
| Recovered manuscript marker no longer stale | 13/13 |
| Final workspace validation errors | 0 in 13/13 |
| Median time to safe invalidated state† | 9.6 ms |
| Median recovery bookkeeping† | 15.0 ms |

†Excludes generation of the revised tool output; local proxy diagnostic only.

mechanism development, these intervals are descriptive sensitivity analyses, not external-validation uncertainty. The decomposition is important: AND/OR support semantics explain selective impact identification on this pilot; the transaction layer contributes atomic multi-file state change, rollback, crash recovery, freshness/serialization and auditable history. The integrated validation package passed 206/206 automated tests across 22 modules.

## 6 Discussion and Limitations

### *6.1 Implications*

The current evidence supports a narrower decomposition than a generic claim of "research reliability." *Artifact reliability* concerns whether code executes, artifacts match and recorded results can be reproduced; *registered-state revision integrity* concerns whether a declared dependency change is reflected consistently in persisted claims, decisions and manuscript bindings. Executable-paper systems such as Paper2Agent make the distinction operational: a tool may remain executable while a semantic correction changes the output on which persisted claims depend. The tracked-proxy study shows that the implemented bridge can carry such an old/new change through immutable runs, supersession, explicit stale-evidence detection and recovery, but it does not establish that a native paper agent will register the correct dependency structure. The experiments further separate two mechanisms. AND/OR support semantics determine *which* objects should change under the declared graph; the transaction layer determines whether that multi-object change is applied atomically and audibly. Registration sensitivity shows that neither mechanism protects an omitted or mistyped dependency. Consequently, precision/recall should be reported together with support-set coverage, and any claim about manuscript protection must also report assertion-binding quality. Whether deterministic gating improves human review or native agent behavior remains an empirical question, not a consequence of the state model.

### *6.2 Threats to validity and limitations*

- **Construct and external validity: no independent real-paper-agent result.** Synthetic experiments use generator-authored reference structures. The 13 historical revision identities and changed semantics are genuine, and the tracked proxy executes the ledger

invalidation/recovery lifecycle, but the executable is a deterministic paper-agent proxy rather than a Paper2Agent-generated MCP and the claim/evidence bindings are developer-authored rather than blind human labels. Thus the study does not establish how often real agents register the right dependencies, how native agent outputs change across revisions, or how well the proposed relation vocabulary agrees with independent scientific judgement.

- **Construct assignment.** The machine semantics are operational once $\mathcal{S}(c)$, $R(c)$, $D(c)$, lineage and assertion bindings are supplied; deciding those relations is not machine-validated here. In particular, the distinction between a required premise and a non-epistemic dependency is an annotation decision whose inter-rater reliability remains unmeasured.
- **Semantic warrant and scope.** A support set is a Boolean declaration checked for structure, and provenance-verified does not mean well designed or validly inferred. The schema has no graded support/contradiction, evidence-quality model or representation of statistical dependence between apparently independent support paths. Claim-to-claim and decision-to-claim dependency are absent, and semantic claim edits currently require a new claim record with relations re-established (Supplement S7).
- **Comparator validity.** EG-R is a reconstruction of published mechanism; no original EviGraph system was run head-to-head, and no conclusion about the native system follows. JTMS/ATMS comparisons test correspondence to known semantics rather than competitive novelty.
- **Auditability.** No person has used TRL in the reported experiments. We claim deterministic checking of invariants, digests, bindings and the export gate and tamper evidence against local edits; we do not claim that reviewers find stale evidence faster or miss fewer stale statements (Supplement S7 is a protocol only).
- **Registration and binding acquisition.** Real registration quality and assertion-binding quality are not measured, and automatic relation proposal from manuscript text or run logs is not implemented. These are upstream acquisition problems that bound achievable protection.
- **Internal and statistical validity.** Diagnostics use fixed seeds and one generator configuration. The 10-event historical structural pilot is authored development evidence; its repository-clustered interval quantifies sensitivity within that pilot, not population-level uncertainty. Timing is from one local machine, and the millisecond tracked-proxy transaction times exclude external agent generation and revised computation.
- **Scale and durability.** Scale is measured to $10^5$ triples using full snapshots. Larger or distributed deployments would require a persistent delta/Merkle design and stronger durability/failure models; the reported transaction tests are local, not distributed consensus or storage-device durability tests.
- **Novelty boundary.** Truth maintenance, provenance, argumentation, dependency invalidation and predecessor-anchored activation already exist [10,12,16,27,28]. Support sets are a restricted declared fragment, not an advance on ATMS or Boolean provenance. The claimed contribution is the research-specific composition, operational boundary and registration-aware evaluation, and no universal claim is made over unsurveyed event-sourcing, data-versioning or ML-lineage systems.

Because a provenance layer lowers the cost of tracing but not the cost of judging, it should support, not replace, human review of generated manuscripts (Supplement S9).

## 7 Conclusion

We presented a transactional state-control layer for declared research dependencies. Its formal and implementation contribution is conditional but concrete: given a registered graph, AND/OR support semantics determine graph-relative liveness, typed invariants constrain legal state, and a predecessor-anchored transaction applies the resulting multi-object revision atomically or not at all. Synthetic mechanism experiments support conformance to that specification and show, within fixtures containing conjunctions and alternatives, why flat singleton edges are insufficient; registration diagnostics show that missing, flattened and spurious relations can each defeat protection. Thirteen genuine historical software revisions were additionally carried through a developer-authored immutable-run proxy lifecycle, and local fault injection exercised rollback and crash recovery. These results establish a mature implementation and a bridge from real semantic changes to persisted revision state, but they do not validate the upstream scientific dependency graph. The remaining publication-critical question is therefore construct and external validity: whether independent experts can reproducibly identify the proposed relations and whether native executable paper agents preserve state more safely under genuine revision. The preregistered old/new study is designed to answer that question with at least 60 revision events from at least 15 repositories, blind annotation, adjudication, strong logical baselines and repository-clustered uncertainty.

**Acknowledgement:** None.

**Funding Statement:** This research received no external funding.

**Author Contributions:** The authors confirm contribution to the paper as follows: study conception and design: Rajesh Kumar, Waqar Ali; methodology and software: Rajesh Kumar, Nikesh Lagun; validation and analysis: Rajesh Kumar, Waqar Ali; investigation and data curation: Rajesh Kumar, Nikesh Lagun; draft manuscript preparation: Rajesh Kumar, Waqar Ali; review and editing: all authors; supervision: Shaoning Zeng. All authors reviewed the results and approved the final version of the manuscript.

**Availability of Data and Materials:** The source code is available at https://github.com/raja21068/ResearchLedger. Seeds, result files, the 13-case tracked-trajectory runner, transaction fault-injection outputs, blind annotation packets and aggregation/adjudication tooling, the confirmatory preregistration and protocols are in the accompanying v2.2 validation package (Supplement S10), for which frozen SHA-256 identifiers are recorded. The historical revisions analysed are drawn from public open-source repositories identified in Supplement S9. The proposed confirmatory old/new paper-agent study has not yet been executed.

**Ethics Approval:** Not applicable. This study did not involve human participants, human data or animals; it analyses only publicly available open-source software repositories and synthetic fixtures. The human-audit study in Supplement S7 is a protocol only and has not been conducted.

**Conflicts of Interest:** The authors declare no conflicts of interest to report regarding the present study.

## References

1. Boiko DA, MacKnight R, Kline B, Gomes G. Autonomous chemical research with large language models. Nature. 2023;624(7992):570-8.
2. Bran AM, Cox S, Schilter O, Baldassari C, White AD, Schwaller P. Augmenting large language models with chemistry tools. Nature Machine Intelligence. 2024;6:525-35.

3. Swanson K, Wu W, Bulaong NL, Pak JE, Zou J. The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies. Nature. 2025.
4. Schmidgall S, Su Y, Wang Z, Sun X, Wu J, Yu X, et al. Agent Laboratory: Using LLM Agents as Research Assistants. In: Findings of the Association for Computational Linguistics: EMNLP 2025; 2025. p. 5977-6043.
5. Miao J, Davis JR, Zhang Y, Pritchard JK, Zou J. Reimagining research papers as interactive and reliable AI agents. Nature. 2026. Available from: https://doi.org/10.1038/s41586-026-11044-y.
6. Meng R, Mishra BD, Chen J, Li CL, Goyal P, Parmar M, et al.. ScientistOne: Towards Human-Level Autonomous Research via Chain-of-Evidence; 2026. Preprint; not peer reviewed. Available from: https://arxiv.org/abs/2605.26340.
7. Ren Z, Li R, Zhang X, Pang Z, Ren S, Zhang J. EviGraph: Evidence-Guided Autonomous Research Agents; 2026. Preprint; not peer reviewed. Available from: https://arxiv.org/abs/2608.04738.
8. Yang R, Li Y, Li S. ARIS: Autonomous Research via Adversarial Multi-Agent Collaboration; 2026. Preprint; not peer reviewed. Available from: https://arxiv.org/abs/2605.03042.
9. Martin-Boyle A, Leckey CAC, Brown MC, Kaur H. PaperTrail: A Claim-Evidence Interface for Grounding Provenance in LLM-based Scholarly Q&A. In: Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems; 2026. .
10. Doyle J. A Truth Maintenance System. Artificial Intelligence. 1979;12(3):231-72.
11. de Kleer J. An Assumption-Based TMS. Artificial Intelligence. 1986;28(2):127-62.
12. Buneman P, Khanna S, Tan WC. Why and Where: A Characterization of Data Provenance. In: Database Theory — ICDT 2001. vol. 1973 of Lecture Notes in Computer Science; 2001. p. 316-30.
13. Green TJ, Karvounarakis G, Tannen V. Provenance Semirings. In: Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems; 2007. p. 31-40.
14. Alchourrón CE, Gärdenfors P, Makinson D. On the Logic of Theory Change: Partial Meet Contraction and Revision Functions. Journal of Symbolic Logic. 1985;50(2):510-30.
15. Toulmin SE. The Uses of Argument. Cambridge University Press; 1958.
16. Dung PM. On the Acceptability of Arguments and its Fundamental Role in Nonmonotonic Reasoning, Logic Programming and n-Person Games. Artificial Intelligence. 1995;77(2):321-57.
17. Modgil S, Prakken H. The ASPIC+ Framework for Structured Argumentation: A Tutorial. Argument & Computation. 2014;5(1):31-62.
18. Moreau L, Missier P. PROV-DM: The PROV Data Model. World Wide Web Consortium; 2013. Available from: https://www.w3.org/TR/prov-dm/.
19. Davidson SB, Freire J. Provenance and Scientific Workflows: Challenges and Opportunities. In: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data; 2008. p. 1345-50.
20. Simmhan YL, Plale B, Gannon D. A Survey of Data Provenance in e-Science. ACM SIGMOD Record. 2005;34(3):31-6.
21. Bechhofer S, Buchan I, De Roure D, Missier P, Ainsworth J, Bhagat J, et al. Why Linked Data is Not Enough for Scientists. Future Generation Computer Systems. 2013;29(2):599-611.
22. Soiland-Reyes S, Sefton P, Crosas M, Castro LJ, Coppens F, Fernández JM, et al. Packaging Research Artefacts with RO-Crate. Data Science. 2022;5(2):97-138.
23. Chirigati F, Rampin R, Shasha D, Freire J. ReproZip: Computational Reproducibility With Ease. In: Proceedings of the 2016 International Conference on Management of Data; 2016. p. 2085-8.
24. Wilkinson MD, Dumontier M, Aalbersberg IJ, Appleton G, Axton M, Baak A, et al. The FAIR Guiding Principles for Scientific Data Management and Stewardship. Scientific Data. 2016;3:160018.
25. Gupta A, Mumick IS. Maintenance of Materialized Views: Problems, Techniques, and Applications. IEEE Data Engineering Bulletin. 1995;18(2):3-19.
26. Kung HT, Robinson JT. On Optimistic Methods for Concurrency Control. ACM Transactions on Database Systems. 1981;6(2):213-26.

27. He J, Yu D. Beyond Memory: A Transactional Continuity Kernel for Long-Lived AI Agents; 2026. Preprint; not peer reviewed. Available from: https://arxiv.org/abs/2608.11632.
28. Feldman SI. Make—a program for maintaining computer programs. Software: Practice and Experience. 1979;9(4):255-65.
29. Mokhov A, Mitchell N, Peyton Jones S. Build systems à la carte. Proceedings of the ACM on Programming Languages. 2018;2(ICFP):79:1-79:29.
30. Acar UA. Self-adjusting computation: an overview. In: Proceedings of the 2009 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation; 2009. p. 1-6.